StringBuffer é uma sequência de caracteres mutável e segura para uso em múltiplas threads, fornecida pelo Java para modificação eficiente de strings no próprio local. É anterior ao StringBuilder e faz parte do pacote java.lang.

Características principais

- Mutabilidade: suporta a modificação da sequência de caracteres sem a criação de novos objetos (anexar, inserir, excluir, substituir, inverter, definirCharAt).
- Segurança de threads: a maioria dos métodos são sincronizados, tornando as operações seguras quando uma única instância de StringBuffer é compartilhada entre várias threads.
- Desempenho: melhor do que a concatenação repetida de Strings (que cria muitos objetos String imutáveis). Geralmente mais lento do que StringBuilder em contextos de thread única devido à sobrecarga de sincronização.
- Capacidade e crescimento automático: armazena internamente caracteres em uma matriz de caracteres redimensionável. A capacidade aumenta automaticamente (novaCapacidade = (capacidadeAntiga * 2) + 2) quando necessário.
- Operações úteis: length(), capacity(), ensureCapacity(int), trimToSize(), toString().

Quando usar StringBuffer

- Utilize quando várias threads modificarem simultaneamente a mesma sequência de caracteres e você precisar de sincronização integrada.
- Prefira StringBuilder em código de thread única (mais rápido, mesma API).
- Para necessidades de strings imutáveis, prefira String ou use toString() em StringBuffer/StringBuilder quando o valor final for necessário.

Exemplo comum (conceitual)

- Criar e modificar:
  - append("text"), insert(index, value), delete(start, end), replace(start, end, "new"), reverse(), setCharAt(index, ch).
- Converter para String: toString().

Compatibilidade e notas

- API introduzida nas primeiras versões do Java e mantida para fins de compatibilidade com versões anteriores.
- Os métodos espelham a API do StringBuilder, exceto pelas implementações sincronizadas.
- Evite sobrecarga de sincronização desnecessária escolhendo StringBuilder quando a segurança de threads não for necessária.

A classe **StringBuffer** em Java é usada para criar e manipular sequências de caracteres **mutáveis** (que podem ser alteradas após a criação) de forma segura em ambientes multithread (**thread-safe**). [1, 2, 3]

Por que usar o StringBuffer?

- **Mutabilidade:** Objetos do tipo `String` comum são imutáveis; cada concatenação ou alteração cria um novo objeto na memória, o que pode causar alto consumo de recursos se feito repetidamente (como dentro de um loop). [1, 2, 3]
- **Thread-Safety (Segurança entre Threads):** Todos os seus métodos são `synchronized`. Isso significa que múltiplas threads podem acessar e modificar o mesmo objeto `StringBuffer` sem corromper os dados. [1, 2, 3, 4]
- **Alternativa de Desempenho:** Se você **não** estiver trabalhando com múltiplas threads, o ideal é usar o **StringBuilder**, que faz a mesma coisa de forma mais rápida por não ter a sobrecarga de sincronização. [1, 2, 3]

Como usar o StringBuffer?

Você instancia a classe, faz as modificações desejadas utilizando métodos como `append()`, `insert()` ou `reverse()`, e depois pode convertê-la de volta para uma `String` usando o método `.toString()`. [1]

Exemplo prático de código:

java

```java
// 1. Criando um StringBuffer
StringBuffer buffer = new StringBuffer("Java");

// 2. Adicionando textos (append)
buffer.append(" Programming");

// 3. Inserindo texto em uma posição específica (índice 4)
buffer.insert(4, " &");

// 4. Substituindo uma parte do texto
buffer.replace(0, 4, "Kotlin");

// 5. Invertendo os caracteres
buffer.reverse();

// 6. Convertendo para String final
String resultado = buffer.toString();
```

Use o código com cuidado.

Principais métodos da classe:

- `append(str)` : Adiciona o conteúdo ao final da sequência.
- `insert(offset, str)` : Insere o texto na posição informada.
- `replace(start, end, str)` : Substitui os caracteres do intervalo especificado.
- `delete(start, end)` : Remove os caracteres entre os índices indicados.
- `reverse()` : Inverte a ordem dos caracteres. [1]